The FloorSet Challenge: Data-Driven SoC Floorplanning

Organizers

Uday Mallappa, AI Research Scientist (Sr. Staff), Intel
Sachin Bhat, AI Solutions Engineer, Intel
Pei Chun Ch'ng, Principal Engineer, Intel
Somdeb Majumdar, Director, AI Lab, Intel
Yuan Lu, PhD Student, UC San Diego
Yusu Wang, Professor, UC San Diego

Background

Fixed-outline floorplanning is an NP-complete combinatorial optimization problem that has been extensively studied in the literature. In industrial System-on-Chip (SoC) design, this problem becomes significantly more complex due to hard constraints on block shapes and locations. The objective is to arrange blocks (also called modules) on a 2D canvas such that a multi-objective cost function is minimized while all constraints are satisfied.

SoC floorplanning differs from traditional bin packing in several key ways. Blocks may have flexible shapes but must adhere to predefined area budgets. Certain blocks are subject to specific placement constraints. Blocks are interconnected through nets and often require connectivity to external terminals on the canvas boundary, creating spatial dependencies that directly influence total wirelength—a primary optimization objective. The cost function is inherently multi-objective, aiming to simultaneously minimize chip bounding-box area and total wirelength while satisfying all placement constraints.

Motivation

Reducing the time-to-convergence for high-complexity designs while navigating multi-dimensional constraint spaces has become a critical bottleneck in the physical design back-end flow. The objective of this competition is to identify highly efficient methodologies capable of addressing these industry-scale challenges.

The primary goal is to shift the design paradigm from manual iterations—which currently span several days—to automated cycles completed within minutes. We hypothesize that this transition is best achieved through machine learning (ML)-guided optimization. However, the effectiveness of such models depends critically on the availability of high-fidelity, labeled datasets. To address this gap, we provide a comprehensive suite of benchmark datasets derived from realistic industrial scenarios. By releasing these

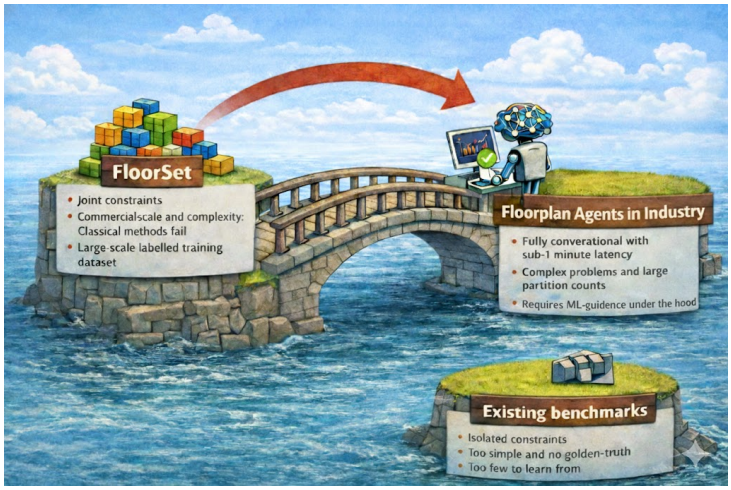

Figure 1: This contest is the catalyst for agentic systems that can solve industrial floorplanning problems in seconds.

datasets (e.g., FloorSet), we challenge the community to develop ML-driven solutions that scale to industrial requirements.

Our underlying hypothesis is that ML guidance enables rapid exploration of the solution space, serving as a foundational component for agentic AI and conversational floorplanning agents. We envision a long-term design environment where architects interact with floorplanning agents via natural language, achieving design iterations with sub-one-minute latency.

Related Work

Fixed-outline floorplanning [6, 14] under diverse placement constraints [17, 25, 41, 42, 43] represents a mature yet increasingly complex challenge within the physical design landscape. While the community has historically relied on legacy benchmarks such as MCNC and GSRC [8, 27] to validate a wide array of solvers—ranging from classical metaheuristics to recent hybrid learning frameworks [1, 2, 5, 7, 22, 23, 34, 36, 39, 40]—these datasets lack the rigor required for modern machine learning validation. Specifically, they fail to provide ground-truth optimal solutions, which are standard in the broader machine learning field, and often treat constraints in isolation rather than integrating them into a holistic design problem. In contrast, our proposed dataset FloorSet [26] bridges this gap by providing millions of layout solutions where area and wirelength are demonstrably optimal, and all placement constraints are strictly satisfied. By leveraging a reverse-engineering methodology that ensures near-optimality by construction, these benchmarks establish a definitive ground-truth reference that enables precise quantification of the optimality gap for any proposed solver. Recognizing that the full industrial complexity of rectilinear partitions and soft blocks poses a steep entry barrier for neural architectures, we introduce FloorSet-Lite. This curated subset focuses exclusively on hard blocks, providing a streamlined yet high-fidelity environment for evaluating the next generation of machine learning-driven EDA tools.

Proof of Concept

Our preliminary experiments indicate that traditional heuristic methods, even with massive parallelism such as distributed SA [28], struggle with the FloorSet-Lite dataset. Even for moderate-sized test cases (e.g., 60 partitions), execution times often exceed 10 minutes without reaching optimality, exhibiting at least a 10% gap in wirelength or area.

Our internal experiments using a diffusion model serves as a counterpoint, achieving high-fidelity solutions in sub-minute intervals. Although the provided dataset is intended for ML training, the contest objective is performance-oriented rather than methodologically restrictive. We invite the application of any algorithmic paradigm, whether purely stochastic, data-driven, or hybrid, that effectively addresses the trade-off between solution quality and runtime.

Problem Statement

Contest Repository

https://github.com/IntelLabs/FloorSet/tree/main/iccad2026contest

Inputs

- $B = \{b_1, b_2, \ldots, b_k\}$ denotes the set of k blocks, where each block b_i is a soft-block, constrained by its predetermined area target from $A = \{a_i \mid i = 1, 2, \ldots, k\}$.
- $T = \{t_1, t_2, \ldots, t_r\}$ represents r terminals, which are predefined fixed points on the 2D plane used for external interfacing. Terminal locations are provided in the input and remain fixed throughout the contest.
- Inter-module connectivity: weighted adjacency matrix $W^{(int)} \in \mathbb{R}^{k \times k}$, where $W^{(int)}_{ij}$ is the weight of the net connecting b_i to b_j. A weight of zero indicates no connection.
- External connectivity: weighted adjacency matrix $W^{(ext)} \in \mathbb{R}^{k \times r}$, where $W^{(ext)}_{ij}$ is the weight of the net connecting b_i to terminal t_j. A weight of zero indicates no connection.
- Soft constraints (for the purpose of this contest, violations incur a penalty but do not disqualify the solution or make it infeasible):
  - **Grouping:** $B^P_{grouping}$ defines P groups of blocks that should be physically abutted (i.e., share a common edge segment of non-zero length, with zero gap). A group is satisfied if all its blocks form a single connected component through shared edges.
  - **Multi-Instantiation Blocks (MIB):** B^Q_{mib} defines Q groups where blocks should share identical dimensions (width and height). These represent instances of the same master cell that must have uniform shape.
  - **Boundary constraints:** $B_{boundary}$ specifies blocks that should be placed such that at least one edge touches the bounding-box boundary (for edge constraints) or such that two edges touch the bounding-box corner (for corner constraints). The specific edge or corner requirement is provided per block in the input.

- Hard constraints (violations that make the solution infeasible for that test case):
  - **Area Targets and Dimensionality:** The dimensions of all blocks must strictly adhere to their specified requirements. For preplaced and fixed-shape blocks, the input dimensions w_i and h_i are immutable. For all other blocks (soft blocks), the realized width w_i and height h_i must satisfy the target area a_i within a 1% relative error threshold:

$$\left| \frac{w_i h_i - a_i}{a_i} \right| \leq 0.01 \tag{1}$$

    Any solution that deviates from the fixed dimensions (for preplaced and fixed-shape blocks) or exceeds the 1% area tolerance for soft blocks is classified as **infeasible**.
  - **Overlap-Free Constraint:** The solution must be strictly overlap-free. For any two distinct blocks b_i and b_j $(i \neq j)$, the area of their intersection must be zero:

$$\text{Area}(b_i \cap b_j) = 0$$

    Any intersection between block geometries, regardless of magnitude, renders the solution **infeasible**. Note: Blocks may share an edge (touch) without overlapping.
  - **Fixed-Shape Immutability:**[1] For each block $b \in B_{\text{fixed}}$, the solution dimensions must exactly match the input specification. The target area a_b is ignored—only the specified width and height matter. A solution is infeasible if, for any fixed-shape block:

$$w_b \neq w_b^{\text{input}} \quad \text{or} \quad h_b \neq h_b^{\text{input}}$$

  - **Preplaced Immutability:**[2] For each block $b \in B_{\text{preplaced}}$, both the location and dimensions must exactly match the input specification. The target area a_b is ignored—only the specified location, width, and height matter. A solution is infeasible if, for any preplaced block:

$$(x_b, y_b) \neq (x_b^{\text{input}}, y_b^{\text{input}}) \quad \text{or} \quad (w_b, h_b) \neq (w_b^{\text{input}}, h_b^{\text{input}})$$

  Infeasible solutions receive a fixed penalty cost of $M = 10$ for that test case, as defined in the Objective Function (Equation 2). This ensures that any feasible solution, regardless of quality, scores better than an infeasible one.

Expected Output

- Overlap-free block locations:

$$L = \{(x_i, y_i) \mid i = 1, 2, \ldots, k\}$$

[1] Previously treated as a soft constraint; updated to hard constraint in both the documentation and the evaluation code.

[2] Previously treated as a soft constraint; updated to hard constraint in both the documentation and the evaluation code.

where (x_i, y_i) are the coordinates of the **lower-left corner** of block b_i. The coordinate system has its origin $(0, 0)$ at the lower-left corner of the canvas, with x increasing to the right and y increasing upward. For preplaced blocks, the coordinates (x_i, y_i) are immutable and must match the input specification exactly.

- Block dimensions:

$$D = \{(w_i, h_i) \mid i = 1, 2, \ldots, k\}$$

  where w_i is the width (extent in the x direction) and h_i is the height (extent in the y direction) of block b_i. Thus, block b_i occupies the rectangular region $[x_i, x_i + w_i] \times [y_i, y_i + h_i]$.

- **Output format:** Solutions must be submitted in the format specified in the FloorSet repository. See the repository documentation for file format details and submission instructions.

Objective Function

We use the following multi-objective cost function:

$$\text{Cost} = \begin{cases} \left(1 + \alpha \cdot (\text{HPWL}^{gap} + \text{Area}^{gap}_{bbox})\right) \times e^{\beta \cdot \text{Violations}^{relative}} \times \max(0.7, \text{RuntimeFactor}^{\gamma}), & \text{if feasible} \\ M, & \text{if infeasible} \end{cases} \tag{2}$$

where:

- A solution is **infeasible** if it violates any hard constraint (block overlap or area tolerance violation). Infeasible solutions receive a fixed penalty cost $M = 10$.
- $\alpha = 0.5$ weights the quality metrics (HPWL and bounding-box area gaps).
- $\beta = 2.0$ controls the exponential violation penalty.
- $\gamma = 0.3$ dampens the runtime factor.
- The $\max(0.7, \cdot)$ caps the runtime benefit at 30%.
- HPWL^{gap} is the **relative gap** between the achieved wirelength and the baseline (optimal) wirelength.
- Area^{gap}_{bbox} is the **relative gap** between the achieved bounding-box area and the baseline (optimal) area.
- $\text{Violations}^{relative} \in [0, 1]$ quantifies soft-constraint violations.
- RuntimeFactor[3] $= \frac{\text{Your Runtime}}{\text{Median Runtime of All Submissions}}$.

[3] This metric is computed independently for each test design, using that individual test case's median runtime as the sole reference point.

Interpretation:

- **Feasible solutions** have costs typically in the range $[0.7, 7.4]$:
  - Best case: perfect quality, 0 violations, fast $\rightarrow \approx 0.7$
  - Worst feasible case: poor quality, 100% soft violations, slow $\rightarrow \approx 7.4 \times 1.5 \approx 11$
- **Infeasible solutions** receive $M = 10$, which is:
  - Higher than any reasonable feasible solution
  - But not so extreme that a single failure dominates the entire score
- **Example impact on Total Score:** With 100 test cases and exponential weighting:
  - A participant who fails 1 large test case (e.g., 120 blocks) will be significantly penalized due to the high weight of larger instances.
  - A participant who fails 1 small test case (e.g., 21 blocks) will be penalized less severely.
- **Exponential violation penalty:** The $e^{\beta V}$ term creates rapidly increasing penalties:

$\text{Violations}^{relative}$	Cost multiplier
0.0	$\times 1.00$
0.25	$\times 1.65$
0.5	$\times 2.72$
1.0	$\times 7.39$

- **Capped runtime factor**[4]**:**

RuntimeFactor	Effect (negative is better)
≤ 0.31 ($\approx 3\times+$ or more faster)	-30% (capped)
0.5 ($2\times$ faster)	-19%
1.0 (median)	baseline
2.0 ($2\times$ slower)	$+23\%$
4.0 ($4\times$ slower)	$+51\%$

- **Example scores:**

Scenario	Cost
Perfect solution, $10\times$ faster	0.70
Perfect solution, $\approx 3\times$ faster	0.70
Perfect solution, median runtime	1.00
10% gaps, 25% violations, $10\times$ faster	1.24
50% soft violations, median runtime	2.72
100% soft violations, $2\times$ slower	9.09
Infeasible (overlap or area violation)	**10.00**

Lower cost is better. Hard constraint violations result in a fixed penalty of $M = 10$, ensuring infeasible solutions are always worse than feasible ones while allowing partial credit across the 100 test cases.

[4]Runtime penalties for slowness are uncapped, while the benefit for speed are subject to a fixed upper limit.

Half-Perimeter Wirelength (HPWL): HPWL sums the weighted Manhattan distances between centroids of connected blocks and terminals. Inter-module HPWL (computed using Manhattan distance between block centroids):

$$\mathrm{HPWL}_{\mathrm{int}} = \sum_{i=1}^{k} \sum_{j>i} W_{ij}^{(\mathrm{int})} \left(\left| c_i^x - c_j^x \right| + \left| c_i^y - c_j^y \right| \right), \tag{3}$$

where $c_i^x = x_i + w_i/2$ and $c_i^y = y_i + h_i/2$ are the centroid coordinates of block b_i.

External connections (computed using Manhattan distance from block center to terminal):

$$\mathrm{HPWL}_{ext} = \sum_{i=1}^{k} \sum_{j=1}^{r} W_{ij}^{(ext)} \left(|x_i + w_i/2 - x_{t_j}| + |y_i + h_i/2 - y_{t_j}| \right).$$

where (x_{t_j}, y_{t_j}) denotes the coordinates of terminal t_j.

Gap-based normalization:

$$\mathrm{HPWL}^{gap} = \frac{(\mathrm{HPWL}_{int} + \mathrm{HPWL}_{ext}) - \mathrm{HPWL}^{baseline}}{\mathrm{HPWL}^{baseline}}$$

A value of 0 means the solution matches the baseline wirelength exactly. A value of 0.15 means the solution is 15% worse than baseline.

Bounding-Box Area: For floorplan $M = \{(x_i, y_i, w_i, h_i) \mid i = 1, \ldots, k\}$:

Lower-left corner of bounding box: $x_{\min} = \min_i x_i$, $y_{\min} = \min_i y_i$.

Upper-right corner of bounding box: $x_{\max} = \max_i(x_i + w_i)$, $y_{\max} = \max_i(y_i + h_i)$.

Let $\mathrm{Area}_{bbox} = (x_{\max} - x_{\min}) \times (y_{\max} - y_{\min})$.

Gap-based normalization:

$$\mathrm{Area}_{bbox}^{gap} = \frac{\mathrm{Area}_{bbox} - \mathrm{Area}_{bbox}^{baseline}}{\mathrm{Area}_{bbox}^{baseline}}$$

A value of 0 means the solution matches the baseline area exactly. A value of 0.05 means the solution uses 5% more area than baseline.

Violation Cost: Violations are computed differently depending on the constraint type:[5]

- **Per-block constraints (Boundary)**: Each block either satisfies (0) or violates (1) its constraint.
- **Per-group constraints** (Grouping, MIB): Violations are counted per group based on the degree of fragmentation (connected components − 1) or shape inconsistency (distinct shapes − 1).

$$\mathrm{Violations}^{relative} = \frac{V_{\mathrm{grouping}} + V_{\mathrm{boundary}} + V_{\mathrm{mib}}}{N_{\mathrm{soft}}} \tag{4}$$

Where N_{soft} is the normalization factor computed as:

$$N_{\mathrm{soft}} = |B_{\mathrm{boundary}}| + \sum_{p=1}^{P} (|G_p| - 1) + \sum_{q=1}^{Q} (|M_q| - 1)$$

[5]Updated to exclude fixed and preplaced from soft constraints.

Here, $|G_p|$ is the number of blocks in grouping group p, and $|M_q|$ is the number of blocks in MIB group q. This normalization ensures that $\text{Violations}^{relative} \in [0, 1]$.

Boundary violations (block does not touch the required edge or corner of the bounding box):

$$V_{\text{boundary}} = \sum_{b \in B_{\text{boundary}}} \mathbb{1}_b, \quad \mathbb{1}_b = \begin{cases} 1, & \text{if } b \text{ does not touch its specified edge or corner} \\ 0, & \text{otherwise.} \end{cases}$$

Grouping violations: Let P be the number of groups, G_p the set of blocks in group p, and c_p the number of connected components formed by blocks in G_p (where two blocks are connected if they share an edge). A perfectly satisfied group has $c_p = 1$. The maximum violation for a group of $|G_p|$ blocks is $|G_p| - 1$ (when all blocks are isolated):

$$V_{\text{grouping}} = \sum_{p=1}^{P} (c_p - 1).$$

Multi-instantiation violations: Let Q be the number of MIB groups, and s_q the number of distinct (w, h) pairs among blocks in group q. A perfectly satisfied MIB group has $s_q = 1$. The maximum violation for a group of $|M_q|$ blocks is $|M_q| - 1$ (when all blocks have different shapes):

$$V_{\text{mib}} = \sum_{q=1}^{Q} (s_q - 1).$$

Dataset

Machine learning has advanced rapidly with scalable transformers leveraging large pre-trained datasets. However, SoC floorplanning lacks such datasets for supervised learning. We provide the FloorSet-Lite dataset (rectangular blocks with a fixed rectangular outline) containing optimal-by-construction layouts:

- **Training:** 1M samples with optimal solutions for sizes ranging from 21 to 120 blocks. Participants may use this data for training ML models or analyzing problem structure. Available at Hugging Face. Use `get_training_dataloader()` from `iccad2026_evaluate.py` for automatic downloading and data access.
- **Validation:** 100 samples (one per size from 21 to 120 blocks), accessible to contestants for validating solution generalizability. Available at Hugging Face. Use `get_validation_dataloader()` from `iccad2026_evaluate.py` for automatic downloading and data access.
- **Test:** 100 samples (one per size from 21 to 120 blocks), hidden from candidates. Used for final submission evaluation.
- The repository includes PyTorch DataLoaders, score evaluators, infeasibility checks, and plotting utilities. **We strongly recommend using the provided evaluator to verify solutions before submission.**
- Baseline values ($\text{HPWL}^{baseline}$, $\text{Area}_{bbox}^{baseline}$) for each test case are provided in the dataset.

Total Score (Cost)

The total score is the weighted average of benchmark costs over 100 test examples (hidden from the candidates), with exponentially increasing weight for larger instances:

$$\text{Total Score} = \sum_{i=21}^{120} \text{Cost}[i] \cdot \frac{e^{n_i}}{\sum_{j=21}^{120} e^{n_j}},$$

where:

- $\text{Cost}[i]$ is the cost (computed using Equation 2) for test case i.
- n_i is the number of blocks in test case i (i.e., $n_i = i$ for this dataset).
- Equivalently, we are multiplying the cost of each testcase i with a normalized weight $\lambda_i = \frac{e^{n_i}}{Z}$, where $Z = \sum_{j=21}^{120} e^{n_j}$ is the normalization constant and $\sum_{i=21}^{120} \lambda_i = 1$:

  $$\text{Total Score} = \sum_{i=21}^{120} \lambda_i \cdot \text{Cost}[i]$$

  This exponential weighting scheme ensures that larger, more challenging instances contribute more heavily to the final score.
- **Lower Total Score is better.** A perfect solution achieving baseline metrics on all test cases with median runtime would have a Total Score close to 0.1.

Baseline metrics and a live leaderboard are provided on the leaderboard page.

Incentivizing Machine Learning Solutions

In this contest, we incentivize data-driven ML-guided solutions in the following ways[6]:

- **Efficiency-Focused Scoring:** Runtime is explicitly integrated into the scoring function, and the exponential weighting by block count penalizes methods that scale poorly with problem size.
- **Scalability Barriers:** Larger instances present significant challenges for classical methods, which struggle to scale efficiently as runtime penalties increase. ML methods that learn from training data can potentially generalize to larger instances more efficiently.

References

[1] S.N. Adya and I.L. Markov. "Fixed-outline floorplanning: enabling hierarchical design". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 11.6 (2003), pp. 1120–1135.

[2] Mohammad Amini et al. "Generalizable Floorplanner through Corner Block List Representation and Hypergraph Embedding". In: KDD '22. Association for Computing Machinery, 2022, pp. 2692–2702. ISBN: 9781450393850.

[6] Submissions found to be reverse-engineering the dataset generator rather than developing genuine algorithmic solutions will be disqualified.

[3] Suchandra Banerjee, Anand Ratna, and Suchismita Roy. "Satisfiability modulo theory based methodology for floorplanning in VLSI circuits". In: *2016 Sixth International Symposium on Embedded Computing and System Design (ISED)*. 2016, pp. 91–95.

[4] Hayward H. Chan, Saurabh N. Adya, and Igor L. Markov. "Are floorplan representations important in digital design?" In: *Proceedings of the 2005 International Symposium on Physical Design*. ISPD '05. San Francisco, California, USA: Association for Computing Machinery, 2005, pp. 129–136. ISBN: 1595930213.

[5] Guolong Chen et al. "VLSI floorplanning based on Particle Swarm Optimization". In: *2008 3rd International Conference on Intelligent System and Knowledge Engineering*. Vol. 1. 2008, pp. 1020–1025.

[6] Tung-Chieh Chen and Yao-Wen Chang. "Modern floorplanning based on B/sup */-tree and fast simulated annealing". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25.4 (2006), pp. 637–650.

[7] Chuan-Wen Chiang. "ANT COLONY OPTIMIZATION FOR VLSI FLOORPLANNING WITH CLUSTERING CONSTRAINTS". In: *Journal of the Chinese Institute of Industrial Engineers* 26.6 (2009), pp. 440–448.

[8] "GSRC". URL: `http://vlsicad.eecs.umich.edu/BK/GSRCbench`.

[9] Zhuolun He et al. "Learn to Floorplan through Acquisition of Effective Local Search Heuristics". In: *2020 IEEE 38th International Conference on Computer Design (ICCD)*. 2020, pp. 324–331.

[10] Xianlong Hong et al. "Corner block list: an effective and efficient topological representation of non-slicing floorplan". In: *IEEE/ACM International Conference on Computer Aided Design. ICCAD - 2000. IEEE/ACM Digest of Technical Papers (Cat. No.00CH37140)*. 2000, pp. 8–12.

[11] Chyi-Shiang Hoo et al. "Variable-Order Ant System for VLSI multiobjective floorplanning". In: *Applied Soft Computing* 13.7 (2013), pp. 3285–3297. ISSN: 1568-4946.

[12] R. Jeyarohini, K. R. Aravind Britto, and M. P. Ramkumar. "Optimization and Representation of Non-Slicing VLSI Floorplanning". In: *2023 4th International Conference on Smart Electronics and Communication (ICOSEC)*. 2023, pp. 26–31.

[13] Pengli Ji et al. "A Quasi-Newton-based Floorplanner for fixed-outline floorplanning". In: *Computers & Operations Research* 129 (2021), p. 105225.

[14] Andrew B. Kahng. "Classical floorplanning harmful?" In: ISPD '00 (2000), pp. 207–213.

[15] Andrew B. Kahng. "Machine Learning for CAD/EDA: The Road Ahead". In: *IEEE Design & Test* 40.1 (2023), pp. 8–16.

[16] Jae-Gon Kim and Yeong-Dae Kim. "A linear programming-based algorithm for floorplanning in VLSI design". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 22.5 (2003), pp. 584–592.

[17] Jianbang Lai et al. "Module placement with boundary constraints using the sequence-pair representation". In: *Proceedings of the ASP-DAC 2001. Asia and South Pacific Design Automation Conference 2001 (Cat. No.01EX455)*. 2001, pp. 515–520.

[18] Ximeng Li et al. "PeF: Poisson's Equation-Based Large-Scale Fixed-Outline Floorplanning". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42.6 (2023), pp. 2002–2015.

[19] Zhu Lichen et al. "An Efficient Simulated Annealing Based VLSI Floorplanning Algorithm for Slicing Structure". In: *2012 International Conference on Computer Science and Service System*. 2012, pp. 326–330.

[20] Jai-Ming Lin and Yao-Wen Chang. "TCG-S: orthogonal coupling of P*-admissible representations for general floorplans". In: *Proceedings 2002 Design Automation Conference (IEEE Cat. No.02CH37324)*. 2002, pp. 842–847.

[21] Jai-Ming Lin and Yao-Wen Chang. "TCG: a transitive closure graph-based representation for non-slicing floorplans". In: *Proceedings of the 38th Design Automation Conference (IEEE Cat. No.01CH37232)*. 2001, pp. 764–769.

[22] Ke Liu et al. "A Hybrid Reinforcement Learning and Genetic Algorithm for VLSI Floorplanning". In: *Proceedings of the 2023 15th International Conference on Machine Learning and Computing*. ICMLC '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 412–418. ISBN: 9781450398411.

[23] Yiting Liu et al. "GraphPlanner: Floorplanning with Graph Neural Network". In: *ACM Trans. Des. Autom. Electron. Syst.* 28.2 (Dec. 2022).

[24] Chaomin Luo, Miguel F. Anjos, and Anthony Vannelli. "Large-scale fixed-outline floorplanning design using convex optimization techniques". In: *Proceedings of the 2008 Asia and South Pacific Design Automation Conference*. ASP-DAC '08. Seoul, Korea: IEEE Computer Society Press, 2008, pp. 198–203. ISBN: 9781424419227.

[25] Yuchun Ma et al. "VLSI floorplanning with boundary constraints based on corner block list". In: *Proceedings of the ASP-DAC 2001. Asia and South Pacific Design Automation Conference 2001 (Cat. No.01EX455)*. 2001, pp. 509–514.

[26] Uday Mallappa et al. "FloorSet - a VLSI Floorplanning Dataset with Design Constraints of Real-World SOCs." In: *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400710773.

[27] "MCNC". URL: `http://vlsicad.eecs.umich.edu/BK/MCNCbench`.

[28] Hesham Mostafa et al. *PARSAC: Fast, Human-quality Floorplanning for Modern SoCs with Complex Design Constraints*. 2024. arXiv: `2405.05495 [cs.OH]`. URL: `https://arxiv.org/abs/2405.05495`.

[29] Hiroshi Murata and Ernest S. Kuh. "Sequence-pair based placement method for hard/soft/pre-placed modules". In: ISPD '98. Monterey, California, USA: Association for Computing Machinery, 1998, pp. 167–172. ISBN: 158113021X.

[30] David Z. Pan. "Closing the Virtuous Cycle of AI for IC and IC for AI". The Council on Electronic Design Automation (CEDA), IEEE. 2021. URL: `https://ieee-ceda.org/presentation/webinar/closing-virtuous-cycle-ai-ic-and-ic-ai`.

[31] Yingxin Pang, Chung-Kuan Cheng, and Takeshi Yoshimura. "An enhanced perturbing algorithm for floorplan design using the O-tree representation". In: *Proceedings of the 2000 International Symposium on Physical Design*. ISPD '00. San Diego, California, USA: Association for Computing Machinery, 2000, pp. 168–173. ISBN: 1581131917.

[32] Martin Rapp et al. "MLCAD: A Survey of Research in Machine Learning for CAD Keynote Paper". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41.10 (2022), pp. 3162–3181.

[33] T. Singha, H.S. Dutta, and M. De. "Optimization of Floor-Planning using Genetic Algorithm". In: *Procedia Technology* 4 (2012). 2nd International Conference on Computer, Communication, Control and Information Technology(C3IT-2012) on February 25 - 26, 2012, pp. 825–829. ISSN: 2212-0173.

[34] Jian Sun et al. *Floorplanning of VLSI by Mixed-Variable Optimization*. 2024. arXiv: `2401.15317 [cs.NE]`.

[35] S. Sutanthavibul, E. Shragowitz, and J.B. Rosen. "An analytical approach to floorplan design and optimization". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 10.6 (1991), pp. 761–769.

[36] Christine L. Valenzuela and Pearl Y. Wang. "A Genetic Algorithm for VLSI Floorplanning". In: *Parallel Problem Solving from Nature PPSN VI*. Ed. by Marc Schoenauer et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 671–680.

[37] D.F. Wong and C.L. Liu. "A New Algorithm for Floorplan Design". In: *23rd ACM/IEEE Design Automation Conference*. 1986, pp. 101–107.

[38] Qi Xu, Song Chen, and Bin Li. "Combining the ant system algorithm and simulated annealing for 3D/2D fixed-outline floorplanning". In: *Appl. Soft Comput.* 40.C (Mar. 2016), pp. 150–160.

[39] Qi Xu et al. "GoodFloorplan: Graph Convolutional Network and Reinforcement Learning-Based Floorplanning". In: *Trans. Comp.-Aided Des. Integ. Cir. Sys.* 41.10 (Oct. 2022), pp. 3492–3502.

[40] Jackey Z. Yan and Chris Chu. "DeFer: Deferred decision making enabled fixed-outline floorplanner". In: *2008 45th ACM/IEEE Design Automation Conference*. 2008, pp. 161–166.

[41] E.F.Y. Young, C.C.N. Chu, and M.L. Ho. "Placement constraints in floorplan design". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 12.7 (2004), pp. 735–745.

[42] F.Y. Young and D.F. Wong. "Slicing floorplans with boundary constraint". In: *Proceedings of the ASP-DAC '99 Asia and South Pacific Design Automation Conference 1999 (Cat. No.99EX198)*. 1999, 17–20 vol.1.

[43] F.Y. Young and D.F. Wong. "Slicing floorplans with pre-placed modules". In: *1998 IEEE/ACM International Conference on Computer-Aided Design. Digest of Technical Papers (IEEE Cat. No.98CB36287)*. 1998, pp. 252–258.

[44] Yong Zhan, Yan Feng, and S.S. Sapatnekar. "A fixed-die floorplanning algorithm using an analytical approach". In: *Asia and South Pacific Conference on Design Automation, 2006*. 2006, 6 pp.-.

[45] Hang Zhao et al. "Online 3D Bin Packing with Constrained Deep Reinforcement Learning". In: *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021*. AAAI Press, 2021, pp. 741–749.

[46] Hai Zhou and Jia Wang. "ACG-adjacent constraint graph for general floorplans". In: *IEEE International Conference on Computer Design: VLSI in Computers and Processors, 2004. ICCD 2004. Proceedings.* 2004, pp. 572–575.